92.5 Pass

Pula Jagadeesh Babu

PDF generated at: 27 Jan 2026 12:17:19 UTC

View this report on HackerRank

Score

92.5% • 185.0 / 200

scored in eClerx - Java Coding GCC Test - 4+ Years in 54 min 21 sec on 27 Jan 2026 16:50:15 IST

Candidate Information

Email	pulajagadeesh.babu@gmail.com
Test	eClerx - Java Coding GCC Test - 4+ Years
Candidate Packet	View
Taken on	27 Jan 2026 16:50:15 IST
Time taken	54 min 21 sec/ 136 min
Work Experience	4 years
Invited by	Vipul
Invited on	23 Jan 2026 11:54:16 IST

Skill Distribution

No.	Skill	Score
1	Java Intermediate	95%
2	SQL Intermediate	0%
3	Java Basic	100%

4	Applied Math	100%
5	Spring Boot Intermediate	100%

Tags Distribution

Medium	94%	Collections	100%
Java	100%	Queues	100%
Stacks	100%	Generics	0%
OOPS	100%	Programming	100%
OOP	100%	Exception Handling	100%
MySQL	0%	Easy	75%
Converter	100%	Control Theory	100%
Diode	0%	Semiconductor	0%
Lambda Expression	100%	Spring Boot	100%
Spring Security	100%	Actuator	100%
Application Metrics	100%		

Questions

Java (Intermediate) • 20 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Empty Collection Multiple Choice	Java (Intermediate)	5/5	-	2 min 28 sec	✓

2	Java Generics Multiple Choice	Java (Intermediate)	0/5	-	28 sec	✗
3	Java Classes Multiple Choice	Java (Intermediate)	5/5	-	27 sec	✓
4	Java PriorityQueue Multiple Choice	Java (Intermediate)	5/5	-	1 min 2 sec	✓
5	Root Class of the Java Exception Heirarchy Multiple Choice	Java (Intermediate)	5/5	-	27 sec	✓

SQL (Intermediate) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	MySQL: Creating a Table Multiple Choice	SQL (Intermediate)	0/5	-	31 sec	✗
7	Java Keywords Multiple Choice	Java (Basic)	5/5	-	24 sec	✓
8	Java Constants Multiple Choice	Java (Basic)	5/5	-	31 sec	✓
9	Buck Converter Proportion Gain Multiple Choice	Applied Math	5/5	-	25 sec	✓
10	Diode Basics and Equivalent Circuits Multiple Choice	-	0/5	-	30 sec	✗

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

11	Java: Currency Exchange Tracker Coding	Java (Intermediate)	75/75	-	15 min 36 sec	✓

Spring Boot (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
12	Spring Boot: Metrics Management & Security Validation Back-end Developer	Spring Boot (Intermediate)	75/75	-	31 min 1 sec	✓

1. Empty Collection

Correct

Multiple Choice | Medium | Collections | Java | Queues | Stacks

Question description

Which code displays an empty (`[]`) collection when executed?

Interviewer guidelines

The correct answer is D.

There are 10 items in the stack initially. In the first loop, we add these 10 items to the stack. In the second loop, we iterate based on the size of the stack, and in every iteration, we remove the item from the top of the stack and insert it into the queue. Then decrement the counter. This is a very important step as this will adjust the index counter and will remove all the items from the stack.
For example, if i is not decremented, then the stack size is reduced and i is incremented. After 4th iteration, the value of the variable I will be greater than the size of the stack and exit from the loop.
But the stack will not be empty. The final elements in the stack are mentioned below:

[5, 6, 7, 8, 9]

To avoid this, we need to decrement the value of the variable i so that it will never increment. It will be always '0'. But the loop will exit when the stack becomes empty as the condition is evaluated to false.

The same case is applicable to the third loop as well as the variable i is required to be reduced. This way it will remove all the items from the queue.

Based on the above explanation, other answers are not correct.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size() - 1; i++) { int n = stack.remove(0); queue.add(n); } for (int i = 0; i < queue.size() - 1; i++) { int n = (int) queue.remove(); i = i - 1; } System.out.println(queue);</code></pre>

- ○ <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); } System.out.println(queue); </code></pre>

- ○ <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); i = i - 1; } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); } System.out.println(queue); </code></pre>

- ● <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); i = i - 1; } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); i = i - 1; } System.out.println(queue); </code></pre>   ✓

No comments.

2. Java Generics

✗ Incorrect

Multiple Choice | Generics | Medium

Question description

What is the result of compiling and/or running this code?

```
public class Generic<T> {
   private T value;
   public Generic(T value) {
      this.value = value;
   }
   public T getValue() {
      return value;
   }
}
```

```
import java.util.ArrayList;
public class Main {
   public static void main(String args[]) {
      ArrayList<Generic>  g = new ArrayList<>();
      Generic<?> g1=new Generic<>(10);
      Generic<?> g2=new Generic<>("Hello");
      g.add(g1);
      g.add(g2);
      int i=g.get(0).getValue();
      String s=g.get(1).getValue();
      System.out.println(s);
      System.out.println(i);
   }
}
```

Interviewer guidelines

In the program, we have created two generic objects with the wildcard ?, which means that the exact type is unknown/could be of any type. However, when retrieving a generic value with a wildcard in Java, we cannot directly assign it to a primitive data type, as primitive data types are not compatible with the Object type.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] Compile-time error while retrieving the values from array list and assigning them to their respective datatypes.

- [ ] Compile-time error while creating the generic object with int value, because generics does not work with primitive data type int

- [x] Hello 10

- [ ] Run-time error because an array list cannot contain generic objects of different types (string and int)

No comments.

---

3. Java Classes

Correct

Multiple Choice | Medium | Java | OOPS | Programming | OOP

Question description

Which statement is true?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] Non-static member classes must have either default or public accessibility.

- [ ] All nested classes can declare static member classes.

Methods in all nested classes can be declared static.

Static member classes can contain non-static methods.

No comments.

4. Java PriorityQueue

Correct

Multiple Choice | Collections | Queues | Medium

Question description

What is the output of the following code?

```
import java.util.*;
public class Main{
   public static void main(String args[]){
      Queue<String> pq = new PriorityQueue<>();
      pq.add("Hello");
      pq.add("Hackerrank");
      String s1=pq.poll();
      pq.remove();
      System.out.println(s1);
      String s2=pq.remove();
      System.out.println(s2);
   }
}
```

Interviewer guidelines

First, we added two elements in the queue, “Hello” and “Hackerrank”. We removed the front element “Hackerrank” of the queue using the poll() method and saved in s1 and print it. We removed the remaining element ”Hello”. Now the queue is empty. When we call the remove() method on an empty queue it throws a “NoSuchElementException” exception. However, the poll() method does not.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Hackerrank NoSuchElementException
- [ ] Hackerrank Hello
- [ ] Hackerrank null
- [ ] Hello Hackerrank

No comments.

---

5. Root Class of the Java Exception Heirarchy

Correct

Multiple Choice | Java | Medium | Exception Handling | Programming

Question description

Which of the following is the root class (apex) of the exception heirarchy in Java?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Throwable
- [ ] Exception
- [ ] Error

- RuntimeException

No comments.

6. MySQL: Creating a Table

Incorrect

Multiple Choice | MySQL | Medium

Question description

Which expression causes a MySQL error?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- CREATE TABLE users (id INT PRIMARY KEY)
- CREATE TABLE IF NOT EXISTS users_backup LIKE users
- **CREATE TABLE IF NOT EXISTS users_backup AS SELECT * FROM users**
- None of the above, all expressions are correct.

No comments.

7. Java Keywords

Correct

Multiple Choice | Easy | Java

Question description

Which keyword is used to take control out of the enclosing loop?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- for
- break ✓
- if
- continue

No comments.

8. Java Constants

Correct

Multiple Choice | Easy | Java

Question description

How is a constant declared so that it can be accessed without making an instance of a class in which it is declared?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- public final int intConst

- public static int intConst
- **public static final int intConst** ✓
- public int intConst

No comments.

9. Buck Converter Proportion Gain

Correct

Multiple Choice | Easy | Converter | Control Theory

Question description

An electrical engineer must optimize a buck converter that powers a sensitive medical device. The engineer must ensure a quick response to load changes without causing instability in the device's power supply. During testing, the engineer considers adjusting the proportional gain of the PID controller used in the converter.

What could be the effect of increasing the proportional gain on the system's response to sudden load changes?

Interviewer guidelines

Increasing the proportional gain of a PID controller can lead to a quicker response to load changes, which is beneficial for maintaining stable output in critical applications like medical devices. However, this adjustment often results in a higher overshoot in the voltage supplied, which could potentially lead to instability. This is particularly critical in medical applications where precise voltage regulation is crucial for device functionality and safety. The engineer must balance the gain settings to optimize response time while minimizing the risk of overshoot and subsequent system instability. In-depth monitoring of system performance during these adjustments will help her achieve the desired stability and ensure the safety of the medical device.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Decreases overshoot

- [x] Increases overshoot
- [ ] Reduces settling time
- [ ] Has no effect on overshoot

No comments.

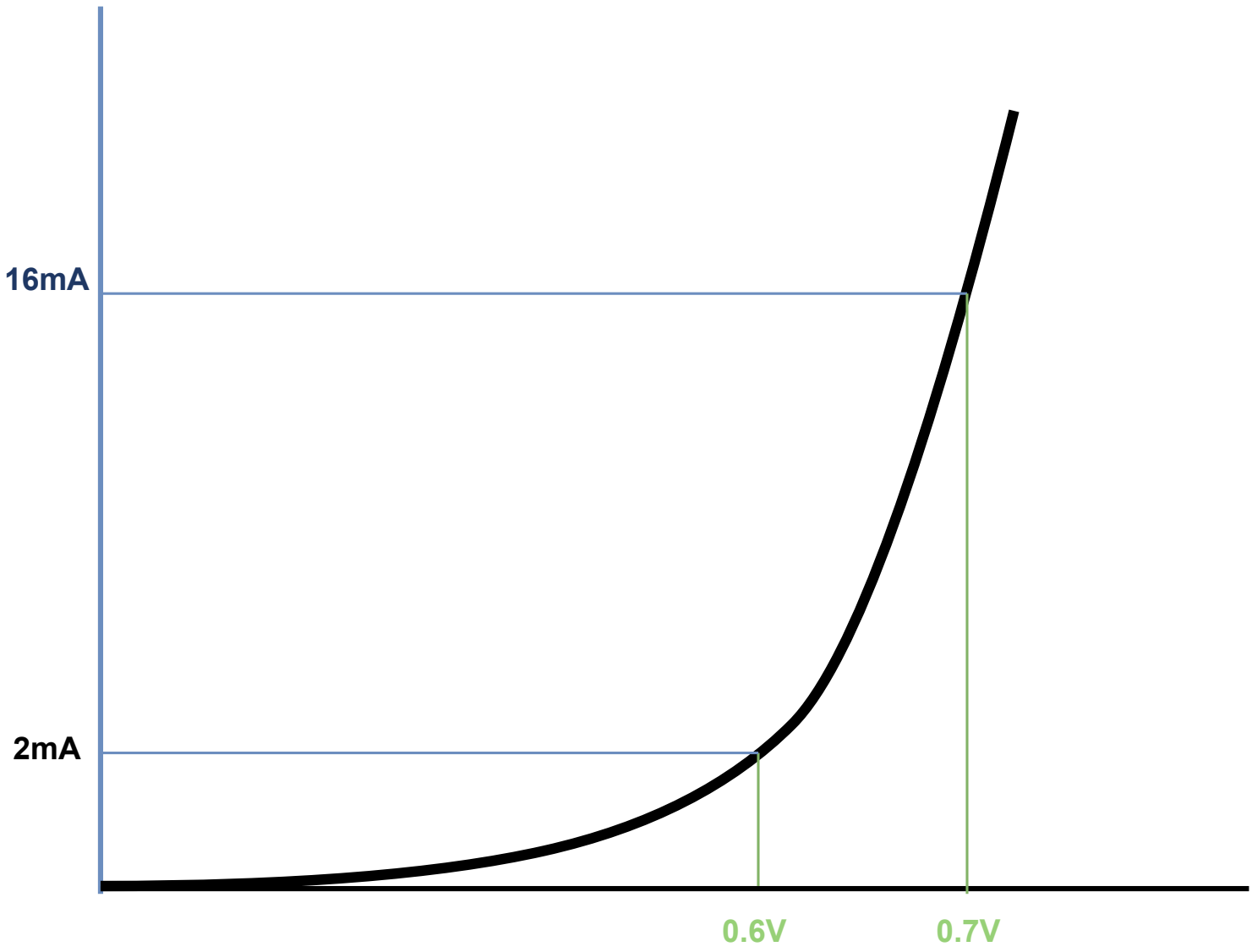
10. Diode Basics and Equivalent Circuits

Incorrect

Multiple Choice | Easy | Diode | Semiconductor

Question description

An engineer is developing a sensor array for an embedded system that includes a new silicon diode among its components. The system relies on precise readings from this array to make real-time decisions through its software. To ensure the diode's performance aligns with the system's requirements, the engineer is provided with the I-V characteristics curve of the diode. The task is to approximate the average AC resistance, r_{av}, over a specified segment of this curve. This resistance calculation is critical as it will directly influence the logic in the system's software, affecting how the embedded system interprets sensor data and makes operational decisions. Using just the curve for reference, determine the *rav* between the marked points. What is the estimate?



Interviewer guidelines

r_{av} is the average AC resistance found between two marked points in the I-V graph. Here r_{av}=0.1/14mA=7Ω.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Not enough details
- ● 11.2 Ohms

6 Ohms

7 Ohms

No comments.

11. Java: Currency Exchange Tracker

Correct

Coding Medium Lambda Expression Java

Question description

Implement a system for financial analysts to filter and process currency exchange rate data.
Create two classes:

1. *ExchangeRateData* class that implements the *IExchangeRateData* interface with:
   - Properties: *currencyPair* (*String*), *rate* (*double*), and *date* (*LocalDate*)
   - Constructor: *exchangeRateData(String currencyPair, double rate, LocalDate date)*
2. *ExchangeRateAnalyser* class that implements the *IExchangeRateAnalyser* interface with:
   - Method: *filterExchangeRates(List exchangeRates, String currencyPair, LocalDate startDate, LocalDate endDate)* that returns the average exchange rate (as a *Double*) for the specified currency pair within the given date range, rounded to two decimal places.

The date range is inclusive, meaning both the start and end dates should be included in the analysis.

Constraints

- The currency pair is a non-empty string in the format "XXX/YYY".
- The exchange rate is a non-negative decimal.
- Dates is in a valid *LocalDate* format.
- The date range is valid with the start date less than or equal to the end date.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer *n*, the number of exchange rate entries.
The next *n* lines each contain an exchange rate entry, that consists of the currency pair, the exchange rate as a decimal, and the date in a format that can be parsed by *LocalDate*. Each of these values is separated by a space.
The line following the exchange rate entries contains the currency pair to filter by.
The next line contains the start date of the date range to filter.

The last line contains the end date of the date range to filter.

All dates in the input are in a consistent format that can be parsed by *LocalDate* - "yyyy-MM-dd".

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
3
USD/EUR 0.82 2024-01-10
USD/EUR 0.84 2024-01-12
USD/EUR 0.86 2024-01-14
USD/EUR
2024-01-10
2024-01-14
```

Sample Output

```
0.84
```

Explanation

There are three exchange rates provided for the "USD/EUR" currency pair. The dates fall within the range from January 10, 2024, to January 14, 2024. The average exchange rate is calculated as (0.82 + 0.84 + 0.86) / 3 = 0.84.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
5
USD/EUR 0.82 2024-01-10
EUR/USD 1.21 2024-01-11
USD/EUR 0.85 2024-01-12
USD/JPY 109.50 2024-01-13
USD/EUR 0.83 2024-01-14
USD/EUR
2024-01-10
2024-01-14
```

Sample Output

```
0.83
```

Explanation

Out of five exchange rates, three of them are for "USD/EUR" and within the date range from January 10, 2024, to January 14, 2024. The average exchange rate for these three entries is calculated as (0.82 + 0.85 + 0.83) / 3 ≈ 0.83.

Interviewer guidelines

▼ SOLUTION

```
class ExchangeRateData implements IExchangeRateData {
    private String currencyPair;
    private double rate;
```

```
    private LocalDate date;

    public ExchangeRateData(String currencyPair, double rate, LocalDate date) {
        this.currencyPair = currencyPair;
        this.rate = rate;
        this.date = date;
    }

    @Override
    public String getCurrencyPair() {
        return currencyPair;
    }

    @Override
    public double getRate() {
        return rate;
    }

    @Override
    public LocalDate getDate() {
        return date;
    }
}

class ExchangeRateAnalyzer implements IExchangeRateAnalyzer {
    @Override
    public Double filterExchangeRates(List<ExchangeRateData> exchangeRates, String currencyPair, LocalDate startD
ate, LocalDate endDate) {
        return exchangeRates.stream()
                .filter(rate -> rate.getCurrencyPair().equals(currencyPair) && rate.getDate().compareTo(startDate) >= 0 && r
ate.getDate().compareTo(endDate) <= 0)
                .map(ExchangeRateData::getRate)
                .mapToDouble(d -> d)
                .average()
                .orElseThrow(ArithmeticException::new);
    }
}
```

Candidate's Solution

Language used: Java 8

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
```

```

interface IExchangeRateData {
    String getCurrencyPair();
    double getRate();
    LocalDate getDate();
}

interface IExchangeRateAnalyzer {
    Double filterExchangeRates(List<ExchangeRateData> exchangeRates, String currencyPair, LocalDate startDate, LocalDate endDate);
}

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

import org.apache.xml.resolver.apps.resolver;

interface IExchangeRateData {
    String getCurrencyPair();
    double getRate();
    LocalDate getDate();
}

interface IExchangeRateAnalyzer {
    Double filterExchangeRates(List<ExchangeRateData> exchangeRates, String currencyPair, LocalDate startDate, LocalDate endDate);
}


class ExchangeRateData implements IExchangeRateData {
          private String currencyPair;
          private double rate;
          private LocalDate date;

          public ExchangeRateData(String currencyPair, double rate,  LocalDate date){
            this.currencyPair=currencyPair;
            this.rate=rate;
            this.date=date;
          }

          @Override
          public String getCurrencyPair(){
            return currencyPair;
          }
           @Override
          public double getRate(){
```

```
            return rate;
          }

          @Override
          public LocalDate getDate(){
            return date;
          }

}

class ExchangeRateAnalyzer implements IExchangeRateAnalyzer {
       @Override
       public Double filterExchangeRates(List<ExchangeRateData> exchangeRates,  String currencyPair,LocalDate startDate,LocalDate endDate){
        double sum = 0.0;
        int count=0;
        for(ExchangeRateData entry:exchangeRates){
            if(entry.getCurrencyPair().equals(currencyPair)){
                LocalDate d = entry.getDate();
                if((d.isEqual(startDate) || d.isAfter(startDate)) && (d.isEqual(endDate) || d.isBefore(endDate))){
                    sum += entry.getRate();
                    count++;
                }
            }
        }
        if(count ==0) return null;
        double avg = sum/count;
        return Math.round(avg*100.0)/100.0;
       }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        int numEntries = Integer.parseInt(reader.readLine().trim());
        List<ExchangeRateData> exchangeRates = new ArrayList<>();

        for (int i = 0; i < numEntries; i++) {
            String[] entry = reader.readLine().trim().split(" ");
            String currencyPair = entry[0];
            double rate = Double.parseDouble(entry[1]);
            LocalDate date = LocalDate.parse(entry[2], DateTimeFormatter.ofPattern("yyyy-MM-dd"));
            exchangeRates.add(new ExchangeRateData(currencyPair, rate, date));
        }

        String currencyPair = reader.readLine().trim();
        LocalDate startDate = LocalDate.parse(reader.readLine().trim(), DateTimeFormatter.ofPattern("yyyy-MM-dd"));
        LocalDate endDate = LocalDate.parse(reader.readLine().trim(), DateTimeFormatter.ofPattern("yyyy-MM-dd"));

```

```
        ExchangeRateAnalyzer analyzer = new ExchangeRateAnalyzer();
        try {
            double averageRate = analyzer.filterExchangeRates(exchangeRates, currencyPair, startDate, endDate);
            System.out.printf("%.2f%n", averageRate);
        } catch (Exception ex) {
            System.out.println("Error: " + ex.getMessage());
        }
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        int numEntries = Integer.parseInt(reader.readLine().trim());
        List<ExchangeRateData> exchangeRates = new ArrayList<>();

        for (int i = 0; i < numEntries; i++) {
            String[] entry = reader.readLine().trim().split(" ");
            String currencyPair = entry[0];
            double rate = Double.parseDouble(entry[1]);
            LocalDate date = LocalDate.parse(entry[2], DateTimeFormatter.ofPattern("yyyy-MM-dd"));
            exchangeRates.add(new ExchangeRateData(currencyPair, rate, date));
        }

        String currencyPair = reader.readLine().trim();
        LocalDate startDate = LocalDate.parse(reader.readLine().trim(), DateTimeFormatter.ofPattern("yyyy-MM-dd"));
        LocalDate endDate = LocalDate.parse(reader.readLine().trim(), DateTimeFormatter.ofPattern("yyyy-MM-dd"));

        ExchangeRateAnalyzer analyzer = new ExchangeRateAnalyzer();
        try {
            double averageRate = analyzer.filterExchangeRates(exchangeRates, currencyPair, startDate, endDate);
            System.out.printf("%.2f%n", averageRate);
        } catch (Exception ex) {
            System.out.println("Error: " + ex.getMessage());
        }
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.1278 sec	32.3 KB
TestCase 1	Easy	Sample	Success	1	0.153 sec	32.5 KB

TestCase 2	Easy	Sample	Success	1	0.1177 sec	32.5 KB
TestCase 3	Medium	Hidden	Success	4	0.1444 sec	34.9 KB
TestCase 4	Medium	Hidden	Success	4	0.1893 sec	36.4 KB
TestCase 5	Medium	Hidden	Success	4	0.1849 sec	36.4 KB
TestCase 6	Medium	Hidden	Success	4	0.1752 sec	35.3 KB
TestCase 7	Medium	Hidden	Success	4	0.1444 sec	36 KB
TestCase 8	Medium	Hidden	Success	4	0.1449 sec	34.9 KB
TestCase 9	Medium	Hidden	Success	8	0.1844 sec	35.9 KB
TestCase 10	Medium	Hidden	Success	8	0.1633 sec	36 KB
TestCase 11	Medium	Hidden	Success	8	0.1722 sec	35.8 KB
TestCase 12	Medium	Hidden	Success	8	0.1514 sec	35.5 KB
TestCase 13	Medium	Hidden	Success	8	0.1689 sec	33.6 KB
TestCase 14	Medium	Hidden	Success	8	0.1663 sec	34.9 KB

No comments.

12. Spring Boot: Metrics Management & Security Validation

Correct

Back-end Developer | Spring Boot | Medium | Spring Security | Actuator | Application Metrics

Question description

There is a tiny student information system built on Spring Boot. The admin wants to know some application metrics and metadata of the system running the application. However, they don't want to reveal to anyone that the application is monitoring information, which means this information should be protected.

The definitions and detailed requirements list follow. You will be graded on whether your application performs data retrieval and manipulation based on given use cases exactly as described in the requirements.

Here are the application requirements:

1. Write a user-defined actuator function that provides the Spring version information of the system. This information should be served via URL, like `/monitoring/springVersion`.
2. Write a custom Spring Boot security function to protect the URL `/monitoring/springVersion`.
3. All the default and system-provided actuator endpoints must be disabled.
4. Allow only the custom actuator endpoint `/monitoring/springVersion`.
5. The rest of the app URLs should work without authentication.

While securing the actuator endpoint, note the following:

- Session management needs to be *stateless*.
- CSRF needs to be disabled.

The application should expose the following endpoints:

`GET` request to `/monitoring/springVersion`:

- this is the actuator endpoint
- it must require basic HTTP authentication to be accessible
- it should return the Spring version used in this app as a response string with status code 200

`GET` request to `/app/student`:

- this is the REST endpoint
- it should be accessible without authentication
- implementation for this has already been provided

The authentication features in this question are handled using the HTTP Basic Authentication. For any given request to be authenticated, the header `"Authorization: Basic {encoded user and password}"` is passed. Generally, in a full-fledged web application, this password would be encoded, but for the scope of this question, the password will be a plain-text string.

Your task is to complete the implementation of the `SpringVersionActuator` & `SpringSecurityConfig` classes so it passes all the test cases when running the provided unit tests.

Note that the username and password combinations have been configured as in-memory values in `SpringSecurityConfig.java`. You just have to handle the part of defining the version actuator endpoint and securing it.

▼ EXAMPLE REQUESTS AND RESPONSES

`GET` request to `/monitoring/springVersion`
Header: Authorization = "Basic admin:admin"
The response code is 200, and the response is:

```
{
  "version": "2.2.1.RELEASE"
}
```

`GET` request to `/monitoring/springVersion`
Header: Authorization = "Basic wrongusername:admin"
The response code is 401, "Unauthorized".

`GET` request to `/actuator/info`
Header: Authorization = "Basic: admin:admin"
The response code is 404, "Not Found".

`GET` request to `/app/student`
The response code is 200, and the response body is `Student`.

Candidate's Submission

Testcase	Test file	Status	Score
shouldGet404SinceNonExposedEndpoints	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
shouldGetResult	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
shouldGetSinceNonActuatorEndpoint	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0

shouldGetAuthorized	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
validateBasePathAndActuatorEndpoint	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0

Review logs: output log
View candidate code

No comments.